UNIT 01- Part 3

Problems and Problem Formulation

In the context of artificial intelligence, an agent must first **formulate a goal** before it can define a **problem. Problem formulation** is the process of deciding which actions and states to consider given that goal.

Problem

A problem is formally defined by four main components:
Initial State: The state the agent starts in (e.g., being in a specific city).
Successor Function (Actions): A description of the possible actions. For any given state x, the function returns a set of (action, successor) pairs reachable from x.
Goal Test: A function that determines whether a given state matches the goal configuration.
Path Cost: A numeric function that assigns a cost to a path. A **solution** is a path from the initial state to a goal state, and an **optimal solution** is the path with the lowest cost

1) Initial state that the agent starts in

For example -

- Consider an agent program **Indian Traveller** developed for travelling from Pune to Chennai travelling through different states. The initial state for this agent can be described as **In (Pune).**

2) A description of the possible actions available to the agent

- The most common formulation uses a successor function. Given a particular state **x, SUCCESSOR Function (X)**returns a set of <action, successor> ordered pairs, where each action is one of the legal actions in state x and each successor is a state that can be reached from x by applying the action.

For example:

From the state **In (Pune)**, the successor function for Indian Traveller problem would return:

{

<Go (Mumbai), In (Mumbai) >

<Go (AhemdNagar), In (AhemdNagar) >

<Go (Solapur), In (Solapur) >

<Go (Satara), In (Satara) >

}

Action

An **action** is a possible move or operation that an agent can perform from a given state.

Example: Indian Traveller Problem

If the agent is in **Pune**, possible actions are:

Go to Mumbai

Go to Ahmednagar

Go to Solapur

Go to Satara

Here, **"Go to Mumbai"** is an action.

Successor

A **successor** is the new state that an agent reaches after performing an action.

Example:

Current state: **In (Pune)**

Action: **Go to Mumbai**

After performing this action, the agent reaches:

Successor state: In (Mumbai)

So,

Action → Go to Mumbai

Successor → In (Mumbai)

Successor Function

A **successor function** gives all possible actions and the states that can be reached by performing those actions from a particular state.
It is represented as:
Successor Function (Current State) →{Action, Successor State}
Example:
For the state **In (Pune):**
Successor(In Pune) ={

<Go Mumbai, In Mumbai>

<Go Ahmednagar, In Ahmednagar>

<Go Solapur, In Solapur>

<Go Satara, In Satara>

}
Meaning:
From Pune, the agent can choose different actions, and each action leads to a new state.

Term	Meaning	Example
Action	A move performed by the agent	Go to Mumbai
Successor	New state reached after action	In Mumbai
Successor Function	Lists all possible actions and their resulting states	From Pune → Mumbai, Ahmednagar, Solapur, Satara

Together, the initial state and successor function implicitly define the **state space** of the problem - which is the set of all states reachable from the initial state.

The state space forms a graph in which the nodes are states and the arcs between nodes are actions.

A path is the state space is a sequence of states connected by a sequence of actions.

3) The goal test

- The goal test determines whether a given state is a goal (final) state. In some problems we can explicitly specify a set of goals. If a particular state is reached, we can check it with the set of goals and if a match is found success can be announced.

For example:

In Indian Traveller problem the goal is to reach Chennai i.e. it is a singleton set:

{In (Chennai)}

In certain types of problems we cannot specify goals explicitly. Instead, goal is specified by an abstract property rather than an explicitly enumerated set of states.

For example:

In chess, the goal is to reach a state called **"Checkmate"** where the opponent's king is under attack and cannot escape.

This **"Checkmate"** situation can be represented using various state spaces.

4) A path cost function

- A path cost function assigns a numeric cost (value) to each path. The problem-solving agent is expected to choose a cost-function that reflects its own performance measure.
- For Indian-Traveller agent, we can have **time required** as cost for path-cost function. It should consider the length of each road being travelled.

Problem Characteristics

To choose an appropriate method for a particular problem it is important to study the problem characteristics which are as follows:

1. Is the problem decomposable to smaller or easier problems?
Can the problem be broken down to smaller problems so that it can be solved independently? Such a decomposed problem can be solved easily.

.2. Can problem solution steps be ignored or undone?

The problem can fall in one of the following three categories:

Ignorable

In this type of problem solution steps can be ignored.

For example:

Theorem proving: In theorem proving if later it is known that certain step is not useful then still one can proceed further as nothing is lost due to repeated steps.

Recoverable

In this type of problem solution steps can be undone and backtracked.

For example: The 8-Puzzle problem. In this problem while moving from initial state to goal state one may make some wrong moves but later can backtrack and recover by making correct move.

Irrecoverable

In this type of problem moves cannot be retracted.

For example:

Playing Chess: In the game of chess one can make wrong move but then this move is neither to be ignored nor it can be recovered.

The above knowledge about the problem helps in determining the control structure. Ignorable problems can be solved using a simple control structure that never backtracks. Recoverable problems can be solved using backtracking. Irrecoverable problems can be solved by recoverable style methods via planning.

3. Is the problem universe predictable?

Further problem can be categorized as problem with certain outcome and problem with uncertain outcome.
uncertain outcome.
In certain outcome problems planning can be done to generate a sequence of operations that lead to a solution. For example, in the **8-Puzzle problem** every time move is made, it is known exactly what will happen. That is there is certain outcome.
In uncertain outcome problem planning can generate a best sequence of operations that can probably lead to problem solution. For example,
in **Bridge game** one cannot know exactly where all the cards are or what the other players will do on their turns. That is there is uncertain outcome.

4. Is a good solution absolute or relative?

Another category of problem is that, when we get solution is it the best one or may be some other path can give optimal (best among all) solution.

For certain problems multiple feasible solutions exist but out of them one is the best and it is the required solution.

Consider following example:

Marcus was a man
Marcus was a Pompeian
Marcus was born in 40 A.D.
All men are mortal.
All Pompeians died when the volcano erupted in 79 A.D.
No mortal lives longer than 150 years.
It is now 2008 A.D.

Question: Is Marcus alive?

Marcus is mortal (1, 4)
Marcus' age is 1964 years. (3, 7)
Marcus is dead. (6, 8, 9)
OR
All Pompeians are died in 79 A.D.
Marcus is dead.

In above problem different reasoning paths lead to the answer. It does not matter which path we follow.

5. Is the problem using consistent knowledge base? Does the problem require lots of knowledge or uses knowledge to constrain solutions?

- To solve problem correctly it is highly necessary that the knowledgebase is consistent on the scale of time and data. In some problems the knowledge base is consistent and in some it is not.

- For example, for evaluating Boolean expression complete knowledge of theorems and laws is required which are always true.
- Whereas data about rate card of hourly labour would change as per time and need to be updated so that manufacturing cost is correctly computed.

State space of the problem

A set of all possible states for a given problem is known as the state space of the problem.

A **state** means a particular condition or situation of a problem at a given time.

State Space Search

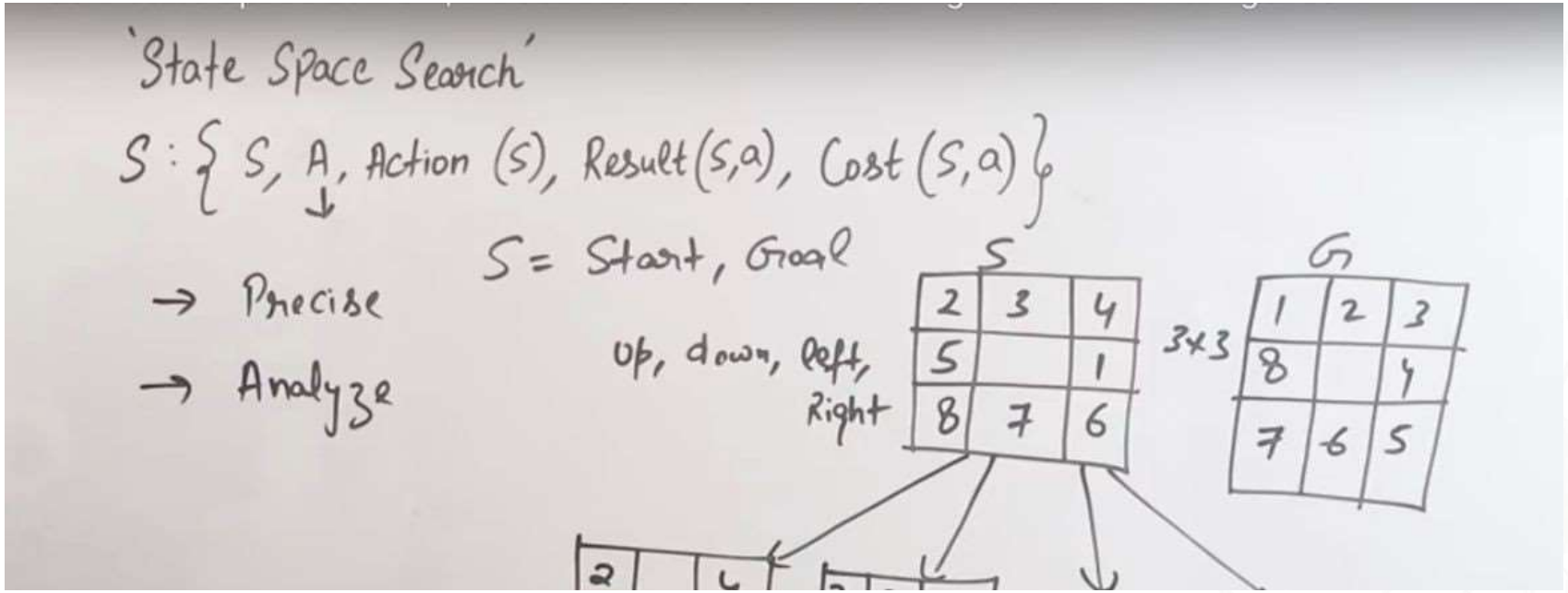
- **State space search** is a method used in Artificial Intelligence to solve problems by exploring different possible states step by step until the goal is reached.
- State space search in AI is a computer science method for finding a solution by looking at several ways an issue might be set up. A "state" is any condition that the agent might find itself in. The "State Space" is the full collection of all conceivable states. The AI needs to use "operators" or actions to get around this area in order to solve an issue. In a maze, the robot's state is where it is right now, and its actions are heading North, South, East, or West. The search stops when the AI finds a state that fits its purpose.

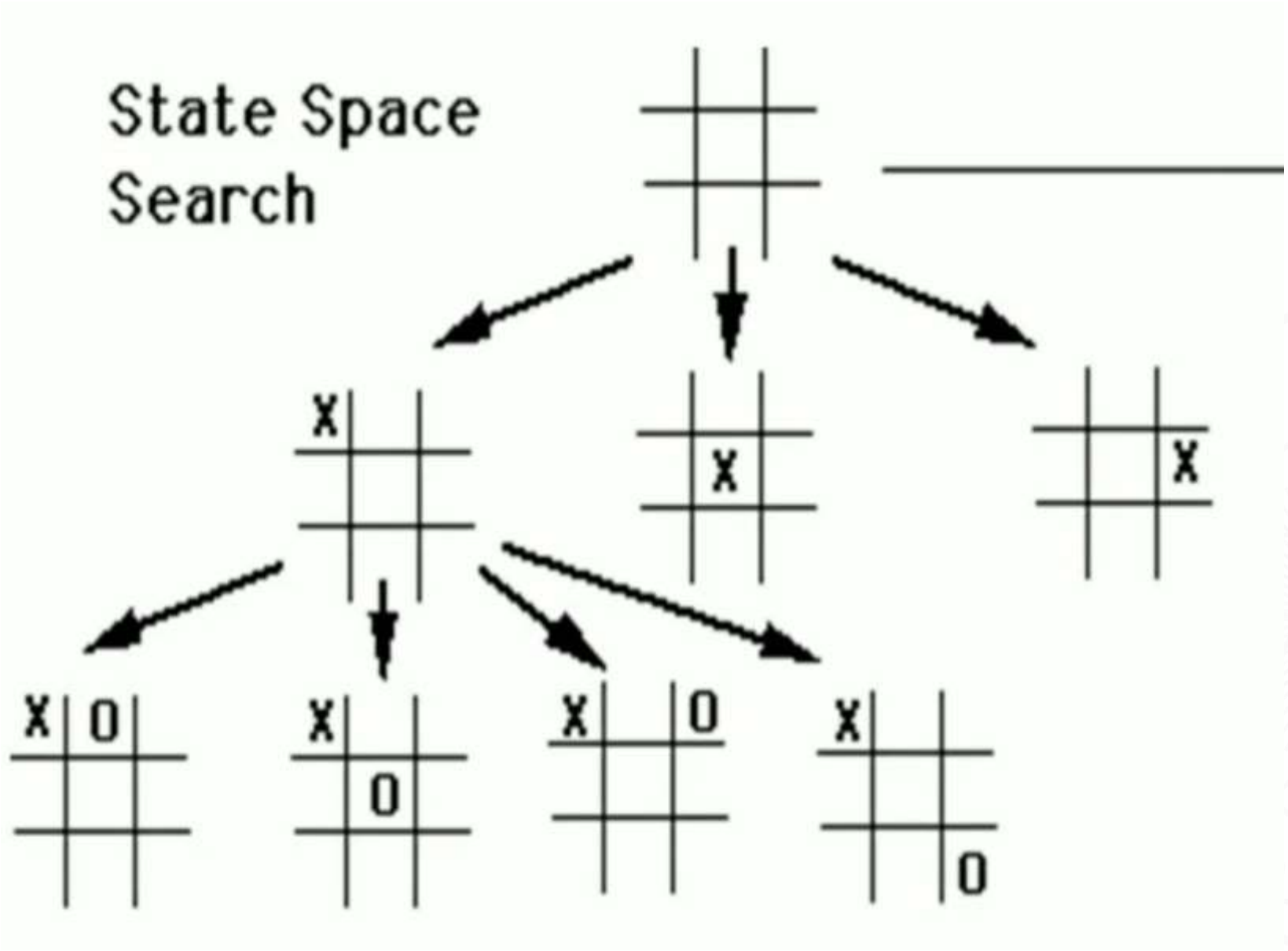
Components of State Space Search

Component	Definition	Role in Search
Initial State	The starting configuration.	Where the search begins.
Actions/Operators	Available moves (e.g., Up, Down).	How the agent moves between states.
Transition Model	Description of what each action does.	Defines the result of an action.
Goal Test	A check to see if a state is the target.	Determines when to stop the search.
Path Cost	The "price" of moving (time/distance).	Helps find the most efficient solution.

Construction of State Space

- The root of search tree is a search node corresponding to initial state. In this state only we can check if goal is reached.
- If goal is not reached we need to consider another state. Such a can be done by expanding from the current state by applying successor function which generates new state. From this we may get multiple states.
- For each one of these, again we need to check goal test or else repeat expansion of each state.
- The choice of which state to expand is determined by the search strategy.
- It is possible that some state, surely, can never lead to goal state. Such a state we need not to expand. This decision is based on various conditions of the problem.


'State Space Search'
S: { S, A, Action (s), Result (s,a), Cost (s,a) }
S = Start, Goal
→ Precise
→ Analyze
Up, down, left, Right
S
G
3×3


State Space
Search
X
X
X
X
O
X
O
X
O
X
O

Start state:

3	10	13	7
9	14	6	1
4		15	2
11	8	5	12

Goal state:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Production system

- A **production system** in Artificial Intelligence is a problem-solving model that uses a set of **rules** to decide what action should be taken in a particular situation.

It is mainly based on **IF–THEN rules.**

Format:

IF condition is true, THEN perform an action.

Example

IF the traffic light is red,

THEN stop the vehicle.

Here:

Condition: Traffic light is red

Action: Stop the vehicle

Main components of a production system

Set of production rules

These are IF–THEN rules used to solve the problem.

Working memory

It stores the current facts or current state of the problem.

Control strategy

It decides which rule should be applied first when more than one rule is possible.

Rule interpreter / inference engine

It checks the rules and applies the suitable rule to reach the solution.

Simple AI example

For diagnosing fever:

Rule 1:

IF temperature is high, THEN the person may have fever.

Rule 2:

IF fever and cough are present, THEN the person may have flu.

So, a production system helps AI make decisions by applying rules step by step.

Issues in the Design of Search Program

1. Performance Evaluation

- A search program should be evaluated using four criteria:
- **Completeness:**
  Whether the search algorithm will find a solution if a solution exists.
- **Optimality:**
  Whether the algorithm finds the best or lowest-cost solution.
- **Time Complexity:**
  How much time the algorithm takes to find the solution.
- **Space Complexity:**
  How much memory the algorithm requires during the search.

2. Exponential Complexity

Search trees grow very fast as the number of choices increases.
The complexity depends on:

- **Branching factor:** Number of possible actions from each state.
- **Depth of goal:** Number of steps needed to reach the goal.
- **Maximum path length:** Longest possible path in the search space.

Because of this, many search problems become difficult to solve using simple search methods.

3. Memory Management

Some search algorithms need a large amount of memory.
For example, **Breadth-First Search** stores all generated nodes, which may quickly fill memory.

To reduce memory usage, methods like:

- **Iterative Deepening Search**
- **Recursive Best-First Search**
- **Simplified Memory-Bounded A***

can be used.

4. Repeated States

A search program may visit the same state again and again.
This causes:

- **Wastage of time**
- **Redundant work**
- **Possibility of infinite loops**

To avoid this, graph search uses a **closed list** to remember already visited states.

5. Accuracy of Problem Representation

The problem should be represented properly in terms of:

- **Initial state**
- **Goal state**
- **Possible actions**
- **Path cost**
- **State space**

Wrong representation may lead to incorrect or inefficient solutions.

6. Choice of Search Strategy

The designer must choose the correct search strategy according to the problem.

Examples:

- **Breadth-First Search** is useful when the shortest path is needed.
- **Depth-First Search** uses less memory but may get stuck.
- **A*** search is useful when a good heuristic is available.